TurtleBot3 Custom Environment Simulation Tutorial

ROS 2 Humble - Ubuntu 22.04

Course Objectives

By the end of this tutorial, students will be able to:

1. Set up and launch TurtleBot3 simulation in Gazebo
2. Visualize robot sensors (camera and IMU) in RViz2
3. Create custom Gazebo worlds with colored walls
4. Implement a vibrating floor sensor using Gazebo plugins
5. Monitor and analyze IMU data for environmental detection

Part 1: TurtleBot3 Setup and Visualization

Prerequisites

- Ubuntu 22.04
- ROS 2 Humble installed
- Gazebo and RViz2 installed

Step 1.1: Install TurtleBot3 Packages

```shell
sudo apt update
sudo apt install ros-humble-turtlebot3*
ros-humble-gazebo-ros-pkgs
```

Checkpoint 1A: Take a screenshot showing successful installation completion.

Step 1.2: Set Environment Variable

```shell
# Set the TurtleBot3 model (waffle_pi has a camera and IMU)
echo 'export TURTLEBOT3_MODEL=waffle_pi' >> ~/.bashrc
source ~/.bashrc

# Verify it's set
echo $TURTLEBOT3_MODEL
```

Expected output: `waffle_pi`

Checkpoint 1B: Run the verification command and take a screenshot of the output.

Step 1.3: Launch TurtleBot3 in Gazebo

```shell
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

This will open Gazebo with the TurtleBot3 Waffle Pi in a simulated world.

Left mouse button moves camer, middle rotates camera, right changes zoom.

Checkpoint 1C: Take a screenshot of Gazebo showing the TurtleBot3 robot in the world.

Step 1.4: Launch RViz2 (Open a NEW terminal)

```shell
source /opt/ros/humble/setup.bash
export TURTLEBOT3_MODEL=waffle_pi
ros2 launch turtlebot3_bringup rviz2.launch.py
```

Step 1.5: Add Camera View in RViz2

In RViz2:

1. Click the "Add" button (bottom left)
2. Select "By topic" tab
3. Expand `/camera/image_raw`
4. Select "Image"
5. Click "OK"

The camera feed should now appear in RViz2!

Checkpoint 1D: Take a screenshot of RViz2 showing both the robot model AND the camera feed.

Step 1.6: Verify Available Topics

Open a **new terminal** and check:

```shell
# List all topics
ros2 topic list

# List camera topics specifically
ros2 topic list | grep camera

# List IMU topics
ros2 topic list | grep imu
```

Checkpoint 1E:

- Run `ros2 topic list > ~/topics_list.txt` to save all topics to a file
- Submit the `topics_list.txt` file

Step 1.7: Test Robot Movement

In a **new terminal**, test robot control:

```shell
ros2 run turtlebot3_teleop teleop_keyboard
```

Use keyboard controls:

- `w` - move forward (increases linear velocity)
- `x` - move backward (decreases linear velocity)
- `a` - turn left (increases angular velocity)
- `d` - turn right (decreases angular velocity)
- `s` - stop (sets all velocity to zero)

Checkpoint 1F: Record a short video (10-15 seconds) showing:

1. The robot moving in Gazebo
2. The camera view changing in RViz2 as the robot moves

Part 2: Custom Environment - Ring World with Red Wall and Vibrating Floor

Step 2.1: Create Custom Workspace Structure

```shell
mkdir -p ~/turtlebot3_custom_ws/worlds
mkdir -p ~/turtlebot3_custom_ws/models
mkdir -p ~/turtlebot3_custom_ws/launch
mkdir -p ~/turtlebot3_custom_ws/scripts
```

Checkpoint 2A: Verify directory structure:

```shell
tree ~/turtlebot3_custom_ws
```

Take a screenshot of the output.

Step 2.2: Create the Ring World File

```shell
cd ~/turtlebot3_custom_ws/worlds
nano ring_world.world
```

Paste the following content:

```xml
<?xml version="1.0"?>
<sdf version="1.6">
  <world name="ring_world">
```

```
    <!-- Physics -->
    <physics type="ode">
      <real_time_update_rate>1000.0</real_time_update_rate>
      <max_step_size>0.001</max_step_size>
      <real_time_factor>1</real_time_factor>
    </physics>

    <!-- A global light source -->
    <include>
      <uri>model://sun</uri>
    </include>

    <!-- Ground plane -->
    <include>
      <uri>model://ground_plane</uri>
    </include>

    <!-- Ring Wall - Front (Normal color) -->
    <model name="wall_front">
      <static>true</static>
      <pose>2.5 0 0.5 0 0 0</pose>
      <link name="link">
        <collision name="collision">
          <geometry>
            <box>
              <size>0.2 5 1</size>
            </box>
          </geometry>
        </collision>
        <visual name="visual">
          <geometry>
            <box>
              <size>0.2 5 1</size>
            </box>
          </geometry>
          <material>
```

```
        <ambient>0.7 0.7 0.7 1</ambient>
        <diffuse>0.7 0.7 0.7 1</diffuse>
      </material>
    </visual>
  </link>
</model>

<!-- Ring Wall - Back (RED) -->
<model name="wall_back_red">
  <static>true</static>
  <pose>-2.5 0 0.5 0 0 0</pose>
  <link name="link">
    <collision name="collision">
      <geometry>
        <box>
          <size>0.2 5 1</size>
        </box>
      </geometry>
    </collision>
    <visual name="visual">
      <geometry>
        <box>
          <size>0.2 5 1</size>
        </box>
      </geometry>
      <material>
        <ambient>1 0 0 1</ambient>
        <diffuse>1 0 0 1</diffuse>
        <specular>0.5 0 0 1</specular>
      </material>
    </visual>
  </link>
</model>

<!-- Ring Wall - Left -->
<model name="wall_left">
```

```
    <static>true</static>
    <pose>0 2.5 0.5 0 0 1.5708</pose>
    <link name="link">
      <collision name="collision">
        <geometry>
          <box>
            <size>0.2 5 1</size>
          </box>
        </geometry>
      </collision>
      <visual name="visual">
        <geometry>
          <box>
            <size>0.2 5 1</size>
          </box>
        </geometry>
        <material>
          <ambient>0.7 0.7 0.7 1</ambient>
          <diffuse>0.7 0.7 0.7 1</diffuse>
        </material>
      </visual>
    </link>
  </model>

  <!-- Ring Wall - Right -->
  <model name="wall_right">
    <static>true</static>
    <pose>0 -2.5 0.5 0 0 1.5708</pose>
    <link name="link">
      <collision name="collision">
        <geometry>
          <box>
            <size>0.2 5 1</size>
          </box>
        </geometry>
      </collision>
```

```
      <visual name="visual">
        <geometry>
          <box>
            <size>0.2 5 1</size>
          </box>
        </geometry>
        <material>
          <ambient>0.7 0.7 0.7 1</ambient>
          <diffuse>0.7 0.7 0.7 1</diffuse>
        </material>
      </visual>
    </link>
  </model>

  <!-- Vibrating Floor Tile near Red Wall -->
  <model name="vibrating_floor">
    <static>false</static>
    <pose>-2 0 0.01 0 0 0</pose>
    <link name="link">
      <inertial>
        <mass>100</mass>
        <inertia>
          <ixx>1.0</ixx>
          <iyy>1.0</iyy>
          <izz>1.0</izz>
        </inertia>
      </inertial>
      <collision name="collision">
        <geometry>
          <box>
            <size>1 1 0.02</size>
          </box>
        </geometry>
      </collision>
      <visual name="visual">
        <geometry>
```

```
            <box>
              <size>1 1 0.02</size>
            </box>
          </geometry>
          <material>
            <ambient>1 1 0 1</ambient>
            <diffuse>1 1 0 1</diffuse>
          </material>
        </visual>
      </link>

      <!-- Plugin to create vibration -->
      <plugin name="vibration_plugin"
filename="libgazebo_ros_force.so">
        <ros>
          <namespace>/vibrating_floor</namespace>
        </ros>
        <link_name>link</link_name>
        <force_x>0</force_x>
        <force_y>0</force_y>
        <force_z>100</force_z>
      </plugin>
    </model>

  </world>
</sdf>
```

Save and exit (Ctrl+X, then Y, then Enter)

Checkpoint 2B: Verify the world file was created:

```shell
ls -lh ~/turtlebot3_custom_ws/worlds/ring_world.world
cat ~/turtlebot3_custom_ws/worlds/ring_world.world | head -20
```

Take a screenshot showing the file exists and the first 20 lines.

Step 2.3: Create Launch File for Custom World

```shell
cd ~/turtlebot3_custom_ws/launch
nano custom_world.launch.py
```

Paste this content:

```python
import os
from ament_index_python.packages import
get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import
PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():

    world_file_name = 'ring_world.world'
    world = os.path.join(os.path.expanduser('~'),
'turtlebot3_custom_ws', 'worlds', world_file_name)

    launch_file_dir =
os.path.join(get_package_share_directory('turtlebot3_gazebo'),
'launch')
    pkg_gazebo_ros = get_package_share_directory('gazebo_ros')

    use_sim_time = LaunchConfiguration('use_sim_time',
default='true')
    x_pose = LaunchConfiguration('x_pose', default='0.0')
    y_pose = LaunchConfiguration('y_pose', default='0.0')
```

```
    gzserver_cmd = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(pkg_gazebo_ros, 'launch',
'gzserver.launch.py')
        ),
        launch_arguments={'world': world}.items()
    )

    gzclient_cmd = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(pkg_gazebo_ros, 'launch',
'gzclient.launch.py')
        )
    )

    robot_state_publisher_cmd = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(launch_file_dir,
'robot_state_publisher.launch.py')
        ),
        launch_arguments={'use_sim_time': use_sim_time}.items()
    )

    spawn_turtlebot_cmd = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(launch_file_dir,
'spawn_turtlebot3.launch.py')
        ),
        launch_arguments={
            'x_pose': x_pose,
            'y_pose': y_pose
        }.items()
    )

    ld = LaunchDescription()
```

```
    ld.add_action(gzserver_cmd)
    ld.add_action(gzclient_cmd)
    ld.add_action(robot_state_publisher_cmd)
    ld.add_action(spawn_turtlebot_cmd)

    return ld
```

Save and exit.

Step 2.4: Create Vibration Generator Script

```shell
cd ~/turtlebot3_custom_ws/scripts
nano vibration_generator.py
```

Paste this:

```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Wrench
import math

class VibrationGenerator(Node):
    def __init__(self):
        super().__init__('vibration_generator')
        self.publisher = self.create_publisher(
            Wrench,
            '/vibrating_floor/gazebo_ros_force',
            10
        )
        self.timer = self.create_timer(0.01,
self.publish_vibration)
```

```
        self.counter = 0
        self.get_logger().info('Vibration generator started!')

    def publish_vibration(self):
        msg = Wrench()
        # Create sinusoidal vibration in Z axis
        frequency = 10.0  # Hz
        amplitude = 50.0  # Force magnitude

        msg.force.z = amplitude * math.sin(2 * math.pi *
frequency * self.counter * 0.01)

        self.publisher.publish(msg)
        self.counter += 1

def main(args=None):
    rclpy.init(args=args)
    node = VibrationGenerator()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Save and exit. Make it executable:

```shell
chmod +x ~/turtlebot3_custom_ws/scripts/vibration_generator.py
```

Checkpoint 2C: Verify the script is executable:

```shell
ls -lh ~/turtlebot3_custom_ws/scripts/vibration_generator.py
```

Take a screenshot showing the execute permissions (should show `-rwxr-xr-x`).

Step 2.5: Launch the Custom Environment

Terminal 1 - Launch Gazebo with custom world:

```shell
export TURTLEBOT3_MODEL=waffle_pi
cd ~/turtlebot3_custom_ws/launch
ros2 launch custom_world.launch.py
```

Checkpoint 2D: Take a screenshot of Gazebo showing:

- The ring of walls (4 walls forming a rectangle)
- One RED wall (back wall)
- The YELLOW vibrating floor tile near the red wall
- The TurtleBot3 robot in the center

Terminal 2 - Start vibration generator:

```shell
cd ~/turtlebot3_custom_ws/scripts
python3 vibration_generator.py
```

Terminal 3 - Launch RViz:

```shell
export TURTLEBOT3_MODEL=waffle_pi
ros2 launch turtlebot3_bringup rviz2.launch.py
```

Add camera view in RViz2 as done in Part 1.

Terminal 4 - Monitor IMU data:

```shell
ros2 topic echo /imu
```

Checkpoint 2E: Take a screenshot showing the IMU data output (any frame of data).

Step 2.6: Test Vibration Detection

Terminal 5 - Control the robot:

```shell
ros2 run turtlebot3_teleop teleop_keyboard
```

Task: Drive the robot to the yellow vibrating floor tile near the red wall.

Checkpoint 2F (CRITICAL - Main Grading Checkpoint):

1. **Before entering the vibrating tile:**

```shell
ros2 topic echo /imu > ~/imu_before_vibration.txt
```

2. Let it record for 5 seconds, then Ctrl+C
3. **While on the vibrating tile:** Drive the robot onto the yellow tile, then:

```shell
ros2 topic echo /imu > ~/imu_on_vibration.txt
```

4. Let it record for 5 seconds, then Ctrl+C
5. **After leaving the vibrating tile:** Drive away from the yellow tile, then:

```shell
ros2 topic echo /imu > ~/imu_after_vibration.txt
```

6. Let it record for 5 seconds, then Ctrl+C

7. **Create a comparison analysis:**

```shell
nano ~/vibration_analysis.txt
```

8. Write your observations (minimum 100 words):

    - Describe the differences in IMU readings
    - Which values changed the most? (linear_acceleration.z, angular_velocity, etc.)
    - Could you detect when the robot entered the vibrating zone?
    - How could this be used in a real robot application?

Submit for grading:

- `imu_before_vibration.txt`
- `imu_on_vibration.txt`
- `imu_after_vibration.txt`
- `vibration_analysis.txt`

Checkpoint 2G: Record a video (30-60 seconds) showing:

1. The robot approaching the red wall
2. The red wall visible in the camera feed (in RViz2)
3. The robot entering the yellow vibrating tile
4. The terminal showing changing IMU values

Part 3: Understanding the System

Step 3.1: Analyze the World Structure

Task: Answer the following questions in a document called `world_analysis.txt`:

1. What is the pose (position and orientation) of the red wall? (Check the world file)
2. What are the dimensions of each wall? (size of the box)
3. What is the position of the vibrating floor relative to the red wall?
4. What color is used for the vibrating floor? (RGB values)
5. What is the frequency and amplitude of the vibration? (Check the Python script)

Checkpoint 3A: Submit `world_analysis.txt` with your answers.

Step 3.2: Topic Investigation

Run these commands and save outputs:

```shell
# Get information about the IMU topic
ros2 topic info /imu --verbose > ~/imu_topic_info.txt

# Get information about camera topic
ros2 topic info /camera/image_raw --verbose >
~/camera_topic_info.txt

# List all active nodes
ros2 node list > ~/nodes_list.txt

# Get info about the vibration generator node
ros2 node info /vibration_generator > ~/vibration_node_info.txt
```

Checkpoint 3B: Submit all four text files:

- `imu_topic_info.txt`
- `camera_topic_info.txt`
- `nodes_list.txt`
- `vibration_node_info.txt`

Grading Rubric

Part 1: Basic Setup (6 points)

- Checkpoint 1A: Installation screenshot (1 points)
- Checkpoint 1B: Environment variable verification (1 points)
- Checkpoint 1C: Gazebo screenshot (1 points)
- Checkpoint 1D: RViz2 with camera screenshot (1 points)
- Checkpoint 1E: Topics list file (1 points)
- Checkpoint 1F: Robot movement video (1 points)

Part 2: Custom Environment (12 points)

- Checkpoint 2A: Directory structure (1 points)
- Checkpoint 2B: World file verification (1 points)
- Checkpoint 2C: Script permissions (1 points)
- Checkpoint 2D: Custom world screenshot (1 points)
- Checkpoint 2E: IMU data screenshot (1 points)
- Checkpoint 2F: Vibration analysis (5 points)
    - Three IMU data files (1 points - 1 each)
    - Analysis document (2 points)
- Checkpoint 2G: Demonstration video (2 points)

Part 3: System Understanding (6 points)

- Checkpoint 3A: World analysis answers (3 points)
- Checkpoint 3B: Topic investigation files (3 points)

Total: 25 points

Submission Instructions

Create a folder named `YourLastName_FirstName_TurtleBot3_Lab` containing:

Required Files:

1. All screenshots (clearly labeled: checkpoint_1A.png, checkpoint_1C.png, etc.)
2. `topics_list.txt`
3. Video files (checkpoint_1F.mp4, checkpoint_2G.mp4)
4. Three IMU data files from checkpoint 2F
5. `vibration_analysis.txt`
6. `world_analysis.txt`
7. Four topic investigation files from checkpoint 3B

Troubleshooting

Issue: Gazebo doesn't launch

Solution:

```shell
killall gzserver gzclient
export TURTLEBOT3_MODEL=waffle_pi
ros2 launch custom_world.launch.py
```

Issue: IMU data shows no vibration

Solution:

- Make sure the vibration_generator.py is running
- Check that the robot is actually on the yellow tile (use camera view)
- Verify the vibration topic: `ros2 topic echo /vibrating_floor/gazebo_ros_force`

Issue: Camera not showing in RViz2

Solution:

- Make sure TURTLEBOT3_MODEL is set to waffle_pi (not burger or waffle)
- Try removing and re-adding the Image display in RViz2
- Check camera topic exists: `ros2 topic list | grep camera`

Issue: Permission denied for vibration_generator.py

Solution:

```shell
chmod +x ~/turtlebot3_custom_ws/scripts/vibration_generator.py
```

Or run with: `python3 vibration_generator.py`

Learning Outcomes Achieved

After completing this tutorial, students have:

✅ Installed and configured ROS 2 packages
✅ Launched and operated Gazebo simulations
✅ Used RViz2 for robot visualization
✅ Created custom Gazebo world files with SDF format
✅ Implemented colored objects in simulation
✅ Created ROS 2 Python nodes for sensor data generation
✅ Monitored and analyzed sensor data (IMU and camera)
✅ Understood robot-environment interaction
✅ Performed data collection and analysis
✅ Documented technical observations

Next Steps / Advanced Challenges

These are optional and just if you want something extra to do.

1. **Challenge 1:** Modify the world to add a blue wall and a second vibrating tile
2. **Challenge 2:** Create a Python node that automatically detects the red wall using camera data
3. **Challenge 3:** Implement autonomous navigation to find and park on the vibrating tile
4. **Challenge 4:** Add a lidar sensor and create a map of the ring world
5. **Challenge 5:** Design a feedback control system that uses IMU data to keep the robot on the vibrating tile

---

References

- ROS 2 Documentation: https://docs.ros.org/en/humble/
- TurtleBot3 Manual: https://emanual.robotis.com/docs/en/platform/turtlebot3/
- Gazebo Tutorials: https://gazebosim.org/docs
- SDF Format: http://sdformat.org/

Tutorial Version: 1.0
Last Updated: 2025
Compatible with: ROS 2 Humble, Ubuntu 22.04, Gazebo 11